



L03: Strings

The Index Train & Text Manipulation

Neon City Cyber Academy

Module 1: Python Basics





Your Team

Emily Chen - *Handler*

"Strings are everywhere - every message, every name, every piece of text. Today you'll master them."

Cole Martinez - *Tech Lead*

"Think of strings as trains. Each character is a car, and we need to know how to access any car we want."



Quick Review

```
name = "Shadow"      # Variable storing text
print(name)          # Output: Shadow

health = 100          # Variable storing number
health -= 20          # Update it
```

Today: Deep dive into TEXT (strings)

What is a String?

A **string** is a sequence of characters.

```
message = "Hello, Agent!"
```

Inside a string:

- Letters: `H`, `e`, `l`, `l`, `o`
- Spaces:
- Punctuation: `,`, `!`
- Numbers (as text!): `"123"` $\neq$ `123`

Everything in quotes is a string.



The Index Train Concept

Think of a string as a **train with numbered cars**:

String: "PYTHON"

Position:	0	1	2	3	4	5
Car:	[P]	[Y]	[T]	[H]	[O]	[N]

Key Rules:

-  **Counting starts at 0** (zero-indexed)
-  Each character has a position number
-  First character is at index **0**
-  Last character is at index **length - 1**



Emily Explains Zero-Indexing:

"Why start at 0? Because computers measure DISTANCE from the start."

"Character 0 = 0 steps from beginning"

"Character 1 = 1 step from beginning"

"It's like floors in Europe: Ground floor = 0, First floor = 1!"

```
word = "CODE"  
# C is 0 steps away = index 0  
# O is 1 step away = index 1  
# D is 2 steps away = index 2  
# E is 3 steps away = index 3
```

Accessing Characters

Use **square brackets** `[]` to get a character:

```
message = "NEON"

print(message[0])    # N
print(message[1])    # E
print(message[2])    # O
print(message[3])    # N
```

Index: The position number

Character: What's at that position

12 34 String Length

Use `len()` to count characters:

```
word = "Python"  
print(len(word))    # 6
```

```
name = "Alex Chen"  
print(len(name))    # 9 (space counts!)
```

```
empty = ""  
print(len(empty))   # 0
```

Length = Total number of characters

⚠ Index Out of Range

```
word = "CAT"
# Indices: 0=C, 1=A, 2=T

print(word[0])    # ✓ C
print(word[2])    # ✓ T
print(word[3])    # ✗ IndexError!
```

Error: `IndexError: string index out of range`

Valid indices: `0` to `len(word) - 1`

For "CAT" (length 3): valid indices are 0, 1, 2



Cole's Pro Tip:

"Last character is ALWAYS at index `len(string) - 1`"

```
message = "Hello"  
# Length = 5  
# Last index = 5 - 1 = 4  
  
print(message[4])    # 'o'
```

"Or use negative indexing - Python's shortcut!"

```
print(message[-1])   # 'o' (last character)
```



BACK

Negative Indexing

Python lets you count **backwards from the end**:

```
word = "CYBER"
```

```
# Forward:    0    1    2    3    4
#             [C] [Y] [B] [E] [R]
# Backward: -5   -4   -3   -2   -1
```

```
print(word[-1])    # R (last)
print(word[-2])    # E (second to last)
print(word[-5])    # C (first)
```

-1 = last, **-2** = second to last, etc.



String Slicing

Get a **substring** (slice of the train):

```
text = "PYTHON"

print(text[0:3])    # PYT (characters 0, 1, 2)
print(text[2:5])    # THO (characters 2, 3, 4)
print(text[1:4])    # YTH
```

Syntax: `string[start:stop]`

- **start:** First index to include
- **stop:** First index to EXCLUDE (stop before this)
- **Result:** characters from `start` to `stop-1`

Emily's Slicing Guide:

"Think of start:stop as 'from here TO (but not including) here' "

```
word = "DETECTIVE"  
#      012345678
```

```
print(word[0:5])    # DETEC (0,1,2,3,4 - stops before 5)  
print(word[3:8])    # ECTIV (3,4,5,6,7 - stops before 8)
```

"The STOP index is exclusive - like a fence you can't cross!"

Slicing Shortcuts

```
text = "ACADEMY"

# From start to index 3
print(text[:3])      # ACA (same as [0:3])

# From index 2 to end
print(text[2:])      # ADEMY (same as [2:7])

# Get everything
print(text[:])       # ACADEMY (full copy)

# Last 3 characters
print(text[-3:])     # EMY
```



String Looping

Strings are **iterable** - you can loop through characters:

```
word = "CODE"  
  
for letter in word:  
    print(letter)
```

Output:

```
C  
O  
D  
E
```

Each iteration, **letter** gets the next character.

Finding Characters

Check if a character/substring exists:

```
message = "Neon City"

print("N" in message)      # True
print("City" in message)   # True
print("Mars" in message)   # False
```

Use **in** keyword:

- Returns **True** if found
- Returns **False** if not found
- Case-sensitive!



Cole's Search Tips:

```
text = "Python Programming"

# Case matters!
print("python" in text)      # False (lowercase p)
print("Python" in text)     # True

# Spaces matter!
print("Programming" in text) # True
print("Program" in text)     # True
print("Programing" in text)  # False (typo!)
```



String Methods: Upper/Lower

```
message = "Hello Agent"

print(message.upper())      # HELLO AGENT
print(message.lower())     # hello agent
print(message)             # Hello Agent (original unchanged)
```

Methods:

- `.upper()` - ALL CAPS
- `.lower()` - all lowercase
- **Original string stays the same** (strings are immutable)



String Methods: Replace

```
text = "I love cats!"

new_text = text.replace("cats", "dogs")
print(new_text)           # I love dogs!
print(text)               # I love cats! (original unchanged)
```

```
code = "1234-5678-9012"
secret = code.replace("-", "")
print(secret)             # 123456789012
```



String Methods: Split

Break a string into a **list** of parts:

```
sentence = "Code is awesome"
words = sentence.split()      # Split on spaces
print(words)                  # ['Code', 'is', 'awesome']
```

```
data = "10,20,30,40"
numbers = data.split(",")    # Split on commas
print(numbers)                # ['10', '20', '30', '40']
```

String Concatenation

Join strings with `+`:

```
first = "Alex"
last = "Chen"

full_name = first + " " + last
print(full_name)           # Alex Chen
```

```
greeting = "Hello, " + "Agent" + "!"
print(greeting)           # Hello, Agent!
```

Note: Can't add string + number directly!

12 34 String Multiplication

Repeat strings with `*`:

```
print("=" * 20)      # =====
print("ABC" * 3)     # ABCABCABC
print("-" * 10)      # -----
```

Use case: Creating borders, patterns, separators

Escape Characters

Special characters using backslash `\`:

```
# New line
print("Line 1\nLine 2")

# Tab
print("Name:\tAlex")

# Quote inside string
print("She said \"Hello!\")
print('It\'s working!')
```

Output:

```
Line 1
Line 2
Name:   Alex
She said "Hello!"
It's working!
```



Emily's Formatting Lesson:

```
name = "Nova"
level = 5
points = 250

# Old way (concatenation)
msg1 = "Agent " + name + " is level " + str(level)

# Better way (f-strings)
msg2 = f"Agent {name} is level {level}"

# F-strings can do math!
msg3 = f"Points: {points * 2}"

print(msg2)      # Agent Nova is level 5
print(msg3)      # Points: 500
```


F-Strings (Formatted Strings)

Most powerful string tool in Python:

```
agent = "Shadow"  
health = 85  
damage = 23  
  
report = f"{agent} took {damage} damage. Health: {health - damage}"  
print(report)  
# Shadow took 23 damage. Health: 62
```

Syntax: Put `f` before quotes, use `{}` for variables/expressions



Challenge: Name Badge

```
first_name = "Alex"
last_name = "Chen"
role = "Junior Detective"

# Create formatted badge
print("=" * 30)
print(f"  {first_name} {last_name}")
print(f"  {role}")
print("=" * 30)
```

Output:

```
=====
  Alex Chen
  Junior Detective
=====
```



Challenge: String Inspector

```
secret = "CODE1234"

print(f"Length: {len(secret)}")
print(f"First character: {secret[0]}")
print(f>Last character: {secret[-1]}")
print(f"First 4 chars: {secret[:4]}")
print(f>Last 4 chars: {secret[-4:]}")
print(f"Contains 'CODE': {'CODE' in secret}")
print(f>All uppercase: {secret.upper()}")
```



Common String Mistakes

```
# ❌ Trying to modify a string
text = "Hello"
text[0] = "Y"      # ERROR! Strings are immutable
```

```
# ✅ Create new string instead
text = "Y" + text[1:]
print(text)        # Yello
```

```
# ❌ Forgetting quotes
name = Alex        # ERROR! Undefined variable
```

```
# ✅ Use quotes for text
name = "Alex"      # Correct!
```



Cole's Final Wisdom:

"Strings are immutable - you can't change them, only create new ones."

```
word = "CAT"
word[0] = "B"    # ❌ Can't do this!

word = "BAT"     # ✅ Can reassign the variable
new_word = "B" + word[1:] # ✅ Can build new string
```

"This is a FEATURE, not a bug - it makes strings safe and predictable!"

✓ Lesson Complete!

You Mastered:

- ✓ Strings as sequence of characters
- ✓ Zero-based indexing (Index Train)
- ✓ Accessing characters with `[]`
- ✓ Negative indexing (-1 for last)
- ✓ Slicing (`[start:stop]`)
- ✓ String methods (upper, lower, replace, split)
- ✓ F-strings for formatting
- ✓ Strings are immutable

Your Assignment

Create a **Message Decoder**:

1. Create a string: `message = "PYTHON IS AWESOME"`
2. Print length
3. Print first and last characters
4. Print middle 6 characters using slicing
5. Convert to lowercase
6. Replace "AWESOME" with "POWERFUL"
7. Split into words
8. Use f-strings to display all results

Bonus: Extract every other character!



See You Next Lesson!

Next Mission: L04 - Nested Constructions

Learn: How to combine operations `int(input())` - boxes inside boxes!